OpenHarmony-Audit-Agent 工具性能说明

本说明记录任务规划与限定函数源码审计的耗时、内存占用和测量方法。测试数据来自本地进程采样，原始记录保存在 reports/performance/ 与 reports/operation-demo/。两组测试分别衡量任务分配开销和实际源码审计成本。

1. 测试环境与统计口径

项目	配置与方法
操作系统	Windows 11，版本 10.0.26200，AMD64
处理器	Intel64 Family 6 Model 197 Stepping 2，GenuineIntel；16 个逻辑 CPU
物理内存	31.50 GiB
规划测试	Python 3.12.14；目标分片数为 2 个 worker
源码审计	Python 3.14.0；Codex CLI 0.163.0-alpha.10.2；配置模型 gpt-6-astra；1 个 worker
内存采样	配置间隔 20 ms；采集父进程与存活子进程 RSS 之和

墙钟时间从被测进程启动统计至退出，包含进程启动和任务执行。规划测试另在 build_plan 循环内部统计单次规划时间。内存指标为采样峰值，本机监控进程与远端模型服务内存不计入；共享页面可能重复计数，短暂峰值和短生命周期子进程可能未被采到。

2. 规划模块实测结果

输入为 1,000、10,000、50,000 个合成 analysis unit，权重按 i mod 7 + 1 生成。每个进程重复规划 20 次，每种规模运行 3 个独立进程，输入规格哈希随原始记录保存。

合成单元数	每进程规划次数	单次规划秒数（3 次运行中位数）	整个进程秒数（中位数）	最大观测 RSS（MiB）
1,000	20	0.000673	0.134	20.06
10,000	20	0.009920	0.320	30.22
50,000	20	0.096826	2.104	68.70

9 个进程均正常退出。该组测试执行任务规划，source_files_scanned、model_calls 和 codeql_calls 均为 0。表中中位数用于描述现有样本；每组 3 个进程的样本量适合观察规划开销，不用于估计尾部延迟或源码审计吞吐量。

3. Parcel 源码审计实测结果

测试目标为 OpenHarmony 官方 commonlibrary_c_utils 仓库，commit 为 b807d3dadb7dbb7639fe85b3a5f72da18d318da2。主文件 base/src/parcel.cpp 共 1,686 行。审计范围为 WriteBuffer、EnsureWritableCapacity、WriteDataBytes 三个函数，以及 GetWritableBytes、CalcNewCapacity、GetPadSize、WritePadBytes 四个直接辅助函数；同时查阅 parcel.h 中的声明和成员定义。

指标	实测值
任务类型	bounded-parcel-source-review，限定函数源码审计
运行次数与并发	1 次运行，1 个 worker；顺序检查入口、数据流和保护条件
端到端墙钟时间	129.60 秒
本机进程树采样峰值 RSS	385.45 MiB（404,168,704 字节）
配置采样间隔与采样数	20 ms；4,136 个样本
进程退出码	0
任务状态	SOURCE_REVIEW_COMPLETE；worker 为 COMPLETE
候选数量	0
原始记录	reports/operation-demo/measurement.json、session.json 及审计产物

计时覆盖准备完成后的 workflow run，包括模型请求等待、源码审阅、产物写入和状态收尾。该次运行采用源码审计路径，独立盲验证、CodeQL、编译及 PoC 执行均未启用。单次结果用于描述指定函数范围的运行成本，未计算 P50、P95 或并发扩展比。

4. 测量与复现方法

在源码根目录安装 benchmark 依赖后执行测量命令。实际审计使用有效的源码快照、模型认证及新建运行目录；YOUR_COMMIT 替换为目标版本。

```
python -m pip install -e ".[benchmark]"
python scripts/measure_command.py --output reports/planner-measurement.json --scope synthetic-planner-only --snapshot synthetic-v1 -- python scripts/benchmark_planner.py --units 50000 --iterations 20
python scripts/measure_command.py --output reports/audit-measurement.json --scope source-audit --snapshot YOUR_COMMIT -- python -m openharmony_audit workflow run --run-root reports/run-data
```

measure_command.py 通过参数数组启动子进程，JSON 输出记录墙钟时间、采样峰值 RSS、采样间隔、退出码和硬件配置，stdout.log 保存命令输出。凭据通过认证目录或环境变量提供。独立运行目录用于区分完整审计与已完成任务的状态检查。

全仓评测按目标快照记录文件数、LOC、实际审计范围、排除项、并发数、模型请求量与 token。CodeQL 建库、查询和辅助验证分别计量，冷缓存与热缓存分别记录。端到端时间取任务整体起止时间；完成率统计包含失败和超时，预算耗尽单独标识。

5. 成本控制与适用范围

主体按分析单元分片并保存检查点，通过角色职责和调查范围约束任务规模。CodeQL-AI 聚焦候选中的具体函数或路径。辅助能力的增量成本可在相同快照、模型预算与复核标准下，与主体单独运行进行对照测量。

局部符号执行的成本受分支数量和建模范围影响。当前 path_executor.py 接口包含 timeout=120 参数，探索逻辑使用 LengthLimiter(max_length=2000) 限制路径长度，对应墙钟超时控制尚未实现。实际耗时和内存以外部采样结果为准。

现有数据覆盖合成任务规划和限定函数源码审计。大规模代码库全量扫描、CodeQL 建库与查询、跨版本回归及多 worker 扩展性尚无实测数据，需按相应任务范围独立测量。